The King Hussein School for Information Technology
Department of Computer Science

Algorithms Design and Analysis
Fall 2015

Final Exam

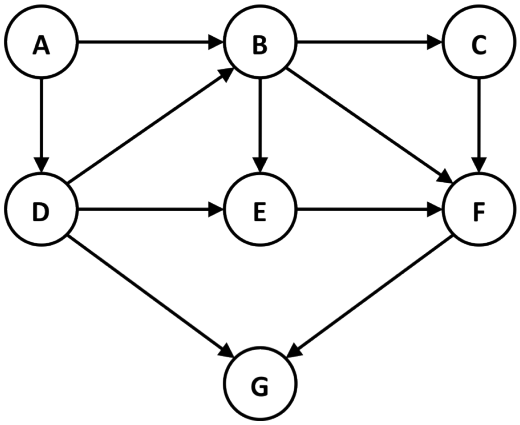
Student Name: Student No:

Question	Points	Score
Q1	21	
Q2	18	
Q3	37	
Q4	24	
Total	**100**	

Date: 10/1/2015

QUESTION 1: Basic Graph Traversals

Consider the following Directed Acyclic Graph:



(1) In what order will the vertices be de-queued from the queue if we apply a **Breadth-First Traversal (BFT)** on the DAG shown above? Follow the **alphabetical order** if there are ties. (7 points)

1st Vertex	2nd Vertex	3rd Vertex	4th Vertex	5th Vertex	6th Vertex	7th Vertex

(2) Apply **Depth-First Traversal (DFT)** on the above DAG. Show the discovery and finish time on each node. Follow the **alphabetical order** if there are ties. (7 points)

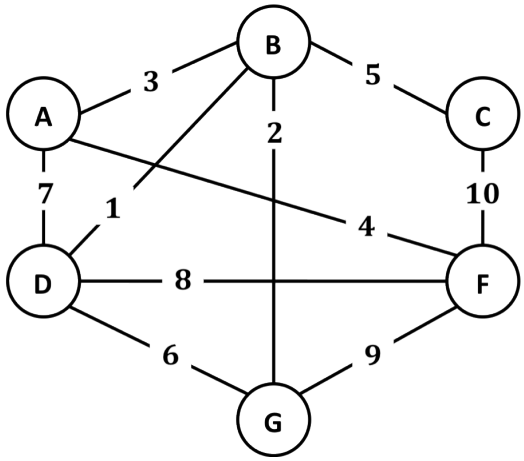
(3) Consider the **Topological Sort** algorithm studied in class, which makes use of the DFT. What is the topological order of the vertices in the above DAG? Assume that **an alphabetical order** is followed during the DFT traversal. (4 points)

1st Vertex	2nd Vertex	3rd Vertex	4th Vertex	5th Vertex	6th Vertex	7th Vertex

(4) How many **bits** are required to represent the above DAG if we use an **Adjacency Matrix** (of bits)? (3 points)

QUESTION 2: Basic Graph Algorithms

Consider the following undirected weighted graph:



(1) In what order will the edges be added to the **Minimal Spanning Tree** if **Kruskal's** algorithm is used on the above graph? Use the edge weight to refer to the edge (for example: use 7 for edge A-D, 3 for edge A-B, etc.) (5 points)

(2) In what order will the edges be added to the **Minimal Spanning Tree** if **Prim's** algorithm is applied on the above graph? Assume that (A) is the source vertex. Use the edge weight to refer to the edge (5 points)

(3) In what order will the vertices be de-queued from the priority queue if **Dijkstra's** algorithm is used for solving the single-source shortest path problem in the above graph? Assume that (A) is the source vertex. (5 points)

1st Vertex	2nd Vertex	3rd Vertex	4th Vertex	5th Vertex	6th Vertex

(4) What is the total number of relaxation attempts (tries) that will be performed if the algorithm of **Bellman-Ford** is applied on the above graph? Provide an exact number. (3 points)

QUESTION 3: Divide and Conquer

Consider the following Divide and Conquer algorithm for computing the **factorial** of ***n***, where ***n*** is a positive number.

```
FACT (N)
      If N = 0 OR N = 1
            Return 1
      Else
            Return N * FACT(N-1)
```

(1) Describe how the factorial function behaves using a **recurrence** equation and then solve the recurrence to find the asymptotic complexity of the function **FACT.** Show your work. (7 points)

(2) Consider a variant of the **Merge Sort** algorithm that splits the array into ***k*** sub-arrays of equal sizes. Describe the behavior of the algorithm using a recurrence equation and then solve the recurrence to find the asymptotic complexity of this Merge Sort variant. Assume that merging ***k*** arrays with total size ***n*** is ***O*(*n*).** Use the recursion tree method and show your work. (7 points)

(3) Solve the following recurrence using the master method: (4 points)

$$T(n) = 8T(\frac{n}{n}) + \Theta(n^3)$$

Recall the **Randomized Selection** algorithm studied in class. Assume that we would like to design a new algorithm called **Randomized Sort** which uses the Randomized Selection algorithm to sort ***n*** elements. Randomized Sort works by calling the Randomized Select algorithm to find the first smallest element, then calls it again to find the second smallest, then the third and so on. Whenever the kth smallest element is found, it is placed in its correct position in a new array. Randomized Sort at the end copies back all of the elements that are in the sorted array into the original array.

(4) Write the pseudo-code of the algorithm described above. (7 points)

(5) Compare between Quick Sort and Randomized Sort. provide a brief justification for each case. (12 points)

	Best Case	**Worst Case**
Quick Sort		
Randomized Sort		

QUESTION 4: Miscellaneous

For each of the following questions, choose the best answer (or answers) as requested.

(1) To check if there is a path between two vertices in a graph with possibly negative edge weights, which of the following algorithms ***should be*** used? **Choose all that applies.**

(A) Depth-First Traversal (B) Breadth-First Traversal (C) Bellman-Ford
(D) Dijkstra (E) None of the provided answers is correct

(2) Which of the following statements is true about the Dynamic Programming algorithm studied in class for finding the **Longest Common Subsequence (LCS)** between two strings ***X*** of length ***M*** and ***Y*** of length ***N***? **Choose all that applies.**

(A) The algorithm runs in **Θ(M+N).**
(B) The algorithm runs in **$\Theta(M^N)$.**
(C) After running the algorithm, the LCS can be extracted from the 2D array in **Θ(M*N).**
(D) After running the algorithm, the LCS can be extracted from the 2D array in **$\Theta(M^N)$.**
(E) None of the above.

(3) Which of the following statements is true about the **Backtracking** solution studied in class for the **0-1 knapsack** problem? **Choose all that applies.**
(Assume ***N*** is the number of items and ***W*** is the capacity of the knapsack)

(A) An exponential number of operations will always be performed.
(B) An exponential number of operations will be performed in the worst case.
(C) If none of the items fits in the knapsack, the number of performed operations will be **O(N).**
(D) If the sum of the item weights is less than ***W***, an exponential number of operations will be performed.
(E) The number of performed operations is **O(N*W)**
(E) None of the above.

(4) Which of the following statements is true regarding **Greedy** Algorithms? **Choose all that applies.**

(A) Greedy algorithms never find the optimal solution. They always find an approximation.
(B) Greedy algorithms always run in **O(N*log*N)** because we need to sort some elements.
(C) There is a Greedy algorithm for the 0-1 knapsack problem.
(D) There is a Greedy algorithm for finding the shortest path in a graph with positive edge weights.
(E) None of the above.

(5) Which of the following statements is true regarding **Dynamic Programming**?

(A) Dynamic Programming is not suitable for Graph problems.

(B) Dynamic Programming always requires creating a 2D array.

(C) Dynamic Programming algorithms are always exponential in time.

(D) Prim's algorithm is an example of a Dynamic Programming algorithm on Graphs.

(E) None of the above.

(6) Assume that we would like to find the **longest path** between two vertices in a directed graph with positive edge weights. To achieve this, all edge weights were multiplied by -1 and a shortest path algorithm was used instead. Which of the following statements is true?

(A) Finding the longest path is never possible. This trick will ***never*** work.

(B) This trick will work if we use Bellman-Ford on an acyclic graph.

(C) This trick will ***always*** work if we use Bellman-Ford.

(D) Dijkstra and Bellman-Ford can be used, but the graph has to be acyclic.

(7) Assume that we would like to find the **shortest path** between two vertices in a directed graph with edge weights that are **possibly negative**. To achieve this, all edge weights were shifted, such that the minimum negative edge weight becomes 0. Which of the following statements is true? **Choose all that apply.**

(A) This trick will work correctly if we use Dijkstra.

(B) This trick will work correctly if we use Bellman-Ford.

(C) This trick will work correctly only if we use Bellman-Ford and there are no negative weight cycles.

(D) This trick will always work correctly.

(E) This trick will never produce a correct answer.

(8) Consider an undirected graph G where each edge e_i has a positive distinct weight w_i. Assume that we have already found the minimum spanning tree T and the shortest path S between two vertices u and v.

Suppose that the graph is modified, such that the weight of each edge increased by 1. Which of the following statements is true?

(A) The edges of T will remain the same and the edges in S will also remain the same.

(B) The edges of T will remain the same but the edges in S may have to be re-computed.

(C) The edges of T may have to be re-computed, but the edges in S will remain the same.

(D) The edges of T and S have to be recomputed.